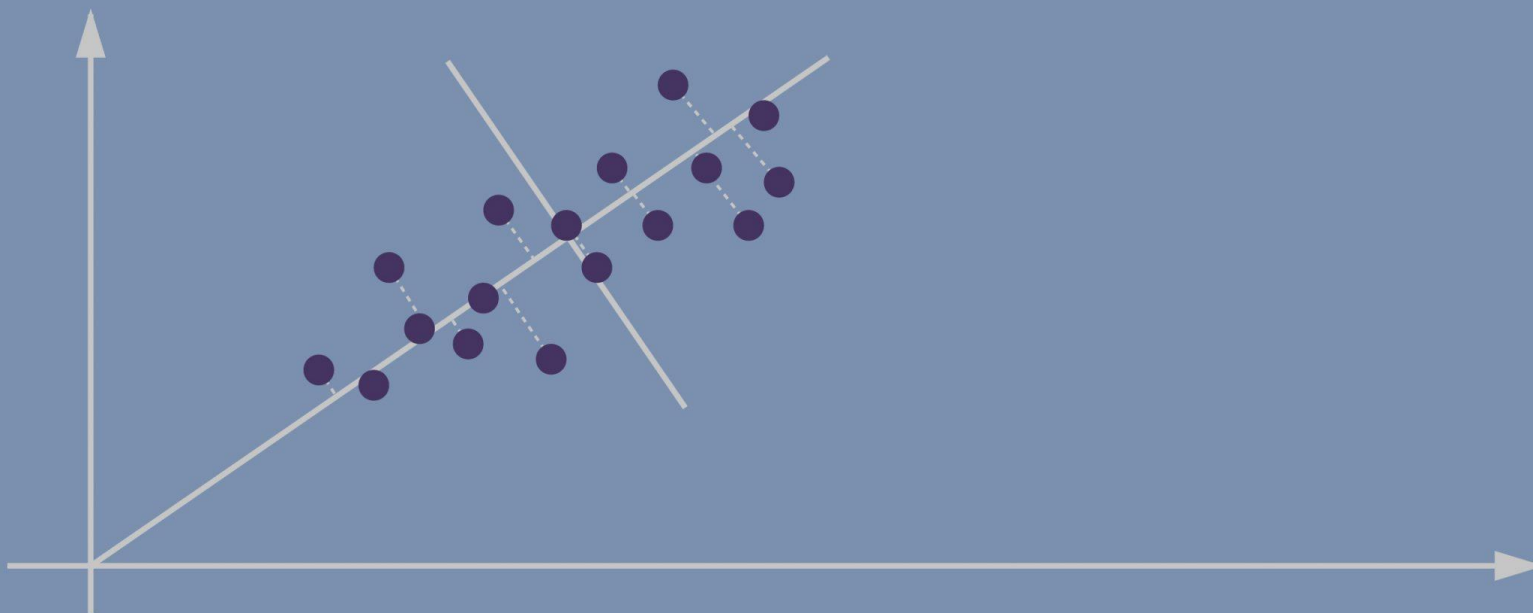




[PCA-06-scaled.jpg \(2560×1051\) \(perfectial.com\)](#)

Lecture 7: Scalable PCA for Dimensionality Reduction

iSheffield Code: **SD-ME-QL**

[COM6012: Scalable ML](#) with Robert Loftin (Slides courtesy of [Haiping Lu](#))

Week 7 Contents / Objectives

- **Recap of Collaborative Filtering**
- Principal Component Analysis (PCA)
- Singular Value Decomposition (SVD)
- Relationship between PCA and SVD
- Scalable PCA and SVD in Spark

Collaborative Filtering

- **Collaborative filtering** algorithms predict the rating that a user will give for an item, **using only past ratings** across users and items.
- CF, unlike content-based systems, does not rely on the descriptive features of the items or users.
- CF can be applied to both **explicit and implicit** ratings
 - explicit: *star ratings, product reviews, like buttons*
 - implicit: *purchases, viewing time, search terms*
- CF faces the **cold-start problem**, where new users and items are added that have no ratings
 - Strategies: *descriptive features, random recommendations*

Collaborative Filtering

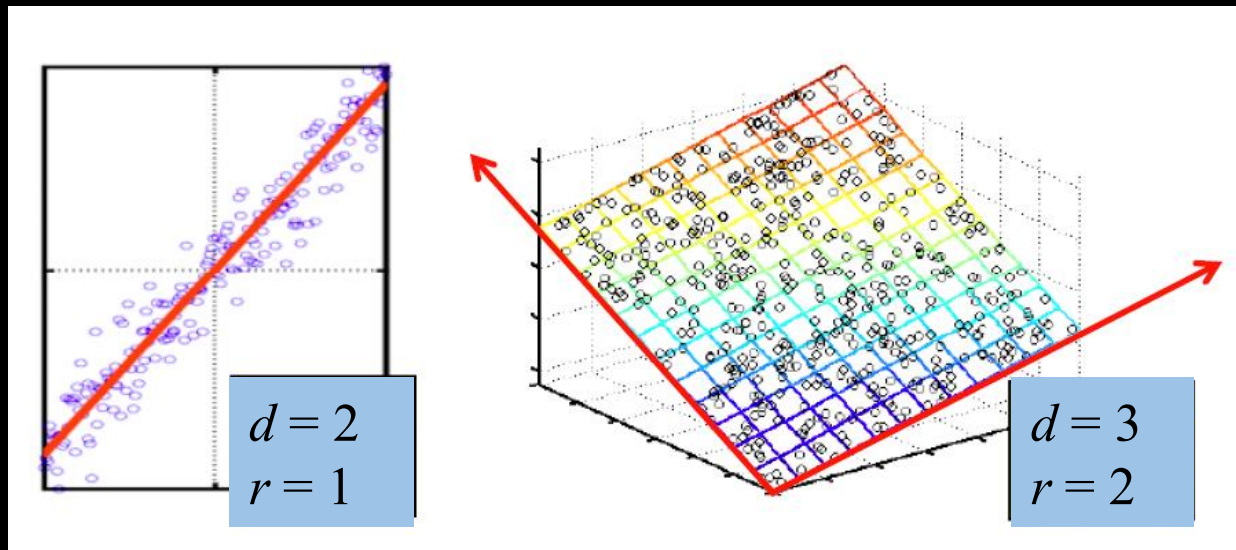
- Matrix-factorization approaches for CF approximate the full rating matrix R from the observed ratings D
- Learns an approximation $R \approx PQ^T$ of this matrix
 - Row p_u of P is a k -dimensional representation of user u in terms of latent semantic features
 - Row q_i of Q is a k -dimensional representation of item u
- Can learn P and Q via Alternating Least Squares (ALS)
 - Fix Q , and solve for each p_u via least squares regression
 - Fix P , and solve for each q_i via least squares regression

Week 7 Contents / Objectives

- Recap of Collaborative Filtering
- **Principal Component Analysis (PCA)**
- Singular Value Decomposition (SVD)
- Relationship between PCA and SVD
- Scalable PCA and SVD in Spark

Dimensionality Reduction

- Raw data: complex and high-dimensional
- **Assumption:** data lie on a low-dimensional subspace
 - Axes of this subspace $\rightarrow$ representation of the data
 - Simpler, more compact, showing interesting patterns



Uses of Dimensionality Reduction

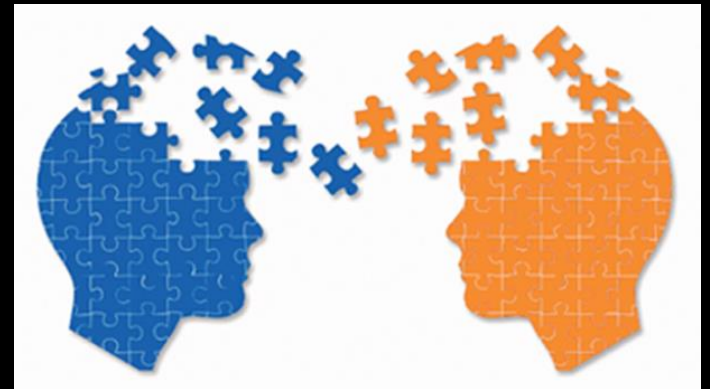
- Discover hidden correlations/concepts/topics
- Remove redundant and/or noisy features
- Easier visualisation and Interpretation
- Easier storage and processing of the data



[owners-icebergs-blog-image-300x300.jpg \(resettogrow.com\)](#)



[1*KvKlx9OnlxdoTfNxWKAY_g.jpeg \(480x320\) \(medium.com\)](#)



[Interpreting and Translation Blog: Image \(wordpress.com\)](#)

Principal Component Analysis

- Input: n data points in a d -dimensional feature space
 - $X_0 \leftarrow n \times d$ raw data matrix, data point $\rightarrow$ row vector x_i
 - $X \leftarrow$ “centered” data – mean of each column is zero
- **Goal:** Find a feature transformation W (a $d \times r$ matrix) such that $T = X W$ preserves important information
- **Idea:** Find W that explains most of the variance of X
 - The first principal component w_1 of X **maximizes the variance** when X is projected onto w_1
 - The k th principal component w_k maximizes the remaining variance after subtracting the variance explained by the first $k - 1$ components

Eigenvectors and Eigenvalues

- For a square matrix M and a vector v , if $Mv = \lambda v$, then v is an **eigenvector** of M , and λ is the corresponding **eigenvalue** of M .
- If M is a real, symmetric $m \times m$ matrix, then M will have a set of m orthogonal eigenvectors.
- Recall that vectors u and v are orthogonal if $u^T v = 0$

PCA $\rightarrow$ Variance Maximisation

- The first principal component w_1 maximises the variance of the transformed data Xw_1
- Mean of X is zero, so we can find w_1 by computing

$$w_1 \in \operatorname{argmax}_w \frac{w^T X^T X w}{\|w\|_2^2}$$

- It turns out, w_1 is an eigenvector corresponding to the largest eigenvalue of $X^T X$

Principal Component Analysis

- We call $X^T X$ the Gramian matrix or the **Scatter matrix**
- The remaining principal components can be found by solving the maximization problem

$$w_{j+1} \in \operatorname{argmax}_w \frac{w^T \hat{X}_j^T \hat{X}_j w}{\|w\|_2^2}$$

- Here $\hat{X}_j^T = X - \sum_{l=1}^j X w_l w_l^T$
- Each PC w_j is the **eigenvector** corresponding to the j th-largest eigenvalue of $X^T X$

Principal Component Analysis

- Input: n data points in a d -dimensional feature space
 - $X_0 \leftarrow n \times d$ data matrix, data point $\rightarrow$ row vector x_i
- Basic PCA algorithm
 - X : **subtract the mean** $\bar{x}$ from each row vector x_i in X_0
 - $X^T X$: compute the Gramian / scatter matrix of X
 - Find the **eigenvectors** and eigenvalues of $X^T X$
 - Construct W ($d \times r$) $\leftarrow$ the **top r** eigenvectors (PCs)
- PCA features $y_i = x_i^T W$ (dimension: $d \rightarrow r$)
 - Zero correlation between features, ordered by variance

Scalability Problems with PCA

- Input dimensionality $\rightarrow$ scatter matrix
 - Images: $100 \times 100 \rightarrow 10^4$; $1000 \times 1000 \rightarrow 10^6$
 - Scatter matrix $X^T X$ is of size d^2
 - $d = 10^4 \rightarrow X^T X$ has 10^8 entries
 - $d = 10^6 \rightarrow X^T X$ has 10^{12} entries
- Computing all d eigenvectors of $X^T X$ takes $O(d^3)$
- Constructing $X^T X$ also takes $O(n^2)$
- Alternative: compute the first r eigenvectors only
 - Efficient incremental eigenvector algorithms available
 - May not need to construct $X^T X$ at all

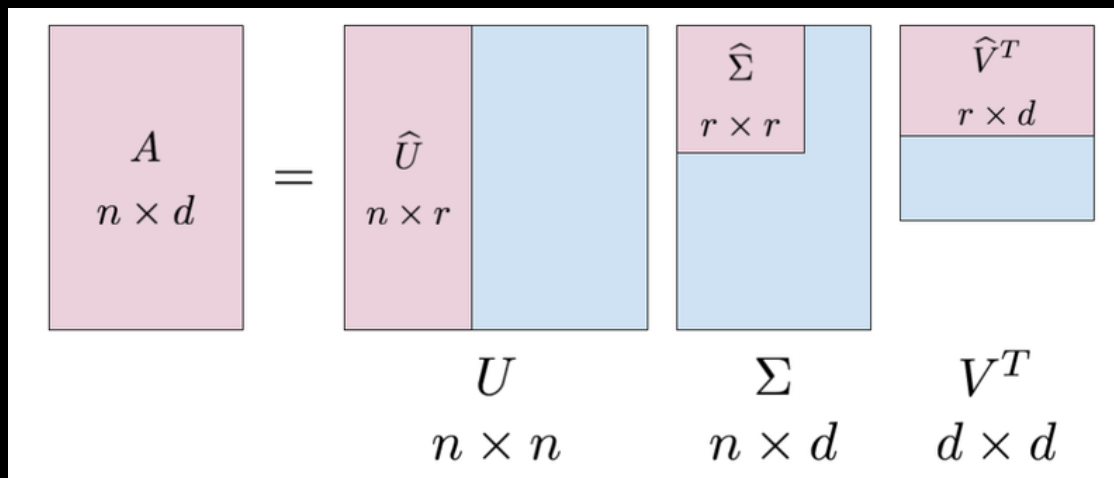
Week 7 Contents / Objectives

- Recap of Collaborative Filtering
- Principal Component Analysis (PCA)
- **Singular Value Decomposition (SVD)**
- Relationship between PCA and SVD
- Scalable PCA and SVD in Spark

Singular Value Decomposition (SVD)

$$A_{[n \times d]} = U_{[n \times r]} \Sigma_{[r \times r]} (V_{[d \times r]})^T$$

- $r \leq d$: the rank of the matrix A
- U : $n \times r$ matrix, column orthonormal, $U^T U = I$
- Σ : $r \times r$ diagonal matrix, strength of each factor
- V : $d \times r$ matrix, column orthonormal, $V^T V = I$



[svd-matrices.png \(800x339\) \(intoli.com\)](#)

Example on a Document x Term

Term Document	data	information	retrieval	brain	lung
CS-TR1	1	1	1	0	0
CS-TR2	2	2	2	0	0
CS-TR3	1	1	1	0	0
CS-TR4	5	5	5	0	0
MED-TR1	0	0	0	2	2
MED-TR2	0	0	0	3	3
MED-TR3	0	0	0	1	1

- $d = 5$ but $r=2 \rightarrow$ two bases $[1\ 1\ 1\ 0\ 0]$ & $[0\ 0\ 0\ 1\ 1]$
- U : document-to-concept similarity matrix
- V : term-to-concept similarity matrix
- Σ : its diagonal elements $\rightarrow$ strength of each concept

Interpretation

Term	data	information	retrieval	brain	lung
Document					
CS-TR1	1	1	1	0	0
CS-TR2	2	2	2	0	0
CS-TR3	1	1	1	0	0
CS-TR4	5	5	5	0	0
MED-TR1	0	0	0	2	2
MED-TR2	0	0	0	3	3
MED-TR3	0	0	0	1	1

retrieval

inf. brain lung

data

CS

MD

doc-to-concept similarity matrix

CS-concept

MD-concept

strength of CS-concept

CS-concept

term-to-concept similarity matrix

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 2 & 2 & 2 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 3 & 3 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.18 & 0 \\ 0.36 & 0 \\ 0.18 & 0 \\ 0.90 & 0 \\ 0 & 0.53 \\ 0 & 0.80 \\ 0 & 0.27 \end{bmatrix} \times \begin{bmatrix} 9.64 & 0 \\ 0 & 5.29 \end{bmatrix} \times \begin{bmatrix} 0.58 & 0.58 & 0.58 & 0 & 0 \\ 0 & 0 & 0 & 0.71 & 0.71 \end{bmatrix}$$

SVD – Dimensionality Reduction

- To reduce the dimensionality further (3 zero singular values have already been removed)
 - Best rank-1 approximation $\rightarrow$

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 2 & 2 & 2 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 3 & 3 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.18 & 0 \\ 0.36 & 0 \\ 0.18 & 0 \\ 0.90 & 0 \\ 0 & 0.53 \\ 0 & 0.80 \\ 0 & 0.27 \end{bmatrix} \times \begin{bmatrix} 9.64 & 0 \\ 0 & 5.29 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 2 & 2 & 2 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The image shows the SVD decomposition of a 7x5 matrix. The original matrix is on the left. It is equal to the product of three matrices. The first matrix is a 7x2 matrix of singular vectors, the second is a 2x2 matrix of singular values, and the third is a 2x5 matrix of right singular vectors. The original matrix and the right singular vector matrix have their last three columns highlighted in yellow, indicating they are zero. The singular value matrix has its diagonal elements highlighted in yellow. The right singular vector matrix has its last three columns highlighted in yellow, indicating they are zero.

Week 7 Contents / Objectives

- Recap of Collaborative Filtering
- Principal Component Analysis (PCA)
- Singular Value Decomposition (SVD)
- **Relationship between PCA and SVD**
- Scalable PCA and SVD in Spark

SVD $\leftrightarrow$ Eigen-decomposition

- SVD of $X = U \Sigma V^T$
- Eigen-decomposition of $X^T X = W \Lambda W^T$
 - Because $X^T X$ is *real* and *symmetric*
- U, V : **orthonormal** $\rightarrow U^T U = I, V^T V = I$
- Σ, Λ : diagonal
- Relationship:
 - $X^T X = V \Sigma^T U^T (U \Sigma V^T) = V \Sigma \Sigma^T V^T = V \Sigma^2 V^T$
 - $X^T X V = (V \Sigma^2 V^T) V = V \Sigma^2$
- Columns of V are eigenvectors of $X^T X$ ($W = V$)
 - Singular values are square roots of eigenvalues ($\Lambda = \Sigma^2$)

PCA via SVD

- Better PCA algorithm:
 - $X_0 \leftarrow n \times d$ data matrix, data point $\rightarrow$ row vector x_i
 - X : subtract mean $\bar{x}$ from each row vector x_i in X_0
 - $U \Sigma V^T \leftarrow$ SVD of X
 - Compute top r **right singular vectors** V of $X \rightarrow$ the PCs
 - The singular values in Σ = the square roots of the eigenvalues of $X^T X$
- We can do this **without** computing the **full eigen-decomposition** of $X^T X$ – *or even constructing $X^T X$*

PCA via SVD

- We can do this **without** computing the **full eigen-decomposition** of $X^T X$ – *or even constructing $X^T X$*
- Key idea is to reduce the problem to repeated calls of the operation $X^T X \mathbf{v}$, for a sequence of d -dimensional vectors $\mathbf{v}$
 - Can compute $X \mathbf{v}$ and $X^T (X \mathbf{v})$ in two separate steps
 - Easily distributed over a large dataset
- Lanczos algorithm uses a version of power iteration to find the r most significant eigenvectors of $X^T X$

Week 7 Contents / Objectives

- Recap of Collaborative Filtering
- Principal Component Analysis (PCA)
- Singular Value Decomposition (SVD)
- Relationship between PCA and SVD
- Scalable PCA and SVD in Spark

PCA APIs in Spark

- DataFrame-based API – [PCA](#) ([source code](#), [Scala doc](#))
 - `pyspark.ml.feature.PCA(k=None, inputCol=None, outputCol=None)`
- RDD-based API – [RowMatrix](#) ([source code](#), [Scala doc](#))
 - `computePrincipalComponents(k)`
 - **Scalable**: `computeSVD(k, computeU=False, rCond=1e-09)`

```
465 @Since("1.6.0")
466 def computePrincipalComponentsAndExplainedVariance(k: Int): (Matrix, Vector) = {
467     val n = numCols().toInt
468     require(k > 0 && k <= n, s"k = $k out of range (0, n = $n]")
469
470     if (n > 65535) {
471         val svd = computeSVD(k)
472         val s = svd.s.toArray.map(eigValue => eigValue * eigValue / (n - 1))
473         val eigenSum = s.sum
474         val explainedVariance = s.map(_ / eigenSum)
```


SVD in Spark MLlib (RDD)

- $U: m \times k$; $\Sigma: k \times k$; $V: m \times k$
- Assumption: n (dimensionality) $< m$ (# samples)
- Different methods based on computational cost:
 - If n is small ($n < 100$) or k is large compared with n ($k > n/2$):
 - Construct $X^T X$ first, then compute its top eigenvalues and eigenvectors **locally** on the driver node
 - Otherwise:
 - Run ARPACK on the driver node to compute eigenvalues/eigenvectors
 - ARPACK makes calls to Spark to compute $(X^T X)v$ – for different vectors v – which in Spark computes in a **distributed** way

Selection of SVD Computation

```
334     if (n < 100 || (k > n / 2 && n <= 15000)) {
335         // If n is small or k is large compared with n, we better compute the Gramian matrix first
336         // and then compute its eigenvalues locally, instead of making multiple passes.
337         if (k < n / 3) {
338             SVDMode.LocalARPACK
339         } else {
340             SVDMode.LocalLAPACK
341         }
342     } else {
343         // If k is small compared with n, we use ARPACK with distributed multiplication.
344         SVDMode.DistARPACK
345     }
346     case "local-svd" => SVDMode.LocalLAPACK
347     case "local-eigs" => SVDMode.LocalARPACK
348     case "dist-eigs" => SVDMode.DistARPACK
349     case _ => throw new IllegalArgumentException(s"Do not support mode $mode.")
```

Acknowledgement & References

- Acknowledgement
 - Some slides are adapted from the [MMDS book](#) slides
- References
 - [Chapter 11](#) of the [MMDS book](#)